

ESP32-S3 Serial Input Processing Code

<https://gemini.google.com/app/f8e984e7320f06b9>

User prompt: i have platformio project with esp32-s3 processor that uses "USBCDC USBSerial;" for a virtual com port. Create a function inside a loop() that collects characters received over the serial terminal and performs following actions: 1. received text "pwd", followed by CR (see point 2 below) types response "Waiting for data" and enables the "data convert and save" flag. 2. CR (return) character starts the action on the collected characters. If data is "pwd", see point 1 above. if data saving flag is set, first run character mapping function (convert collected ascii characters to the raw keyboard layout for the slovenian language, for example character "y" is "z" and "z" is "y", and so on), then print out the received and converted data over serial terminal.

Response: Here is a non-blocking serial processing function for the ESP32-S3 using USBCDC USBSerial.

Complete Implementation

```
#include <Arduino.h>

// Declaration of USBCDC instance if not already initialized elsewhere
#ifdef CONFIG_IDF_TARGET_ESP32S3
#include "USB.h"
#include "USBCDC.h"
extern USBCDC USBSerial;
#endif

// Global state flags and input buffer
bool dataConvertSaveFlag = false;
String inputBuffer = "";

/**
 * Maps input ASCII characters to the Slovenian QWERTZ keyboard layout.
 */
char mapToSlovenianLayout(char c) {
    switch (c) {
        // Core QWERTZ swap
        case 'y': return 'z';
        case 'z': return 'y';
    }
}
```

```

    case 'Y': return 'Z';
    case 'Z': return 'Y';

    // Optional QWERTY -> Slovenian QWERTZ symbol shifts:
    // case '=': return '+';
    // case '-': return '/';
    // case '[': return 'f'; // Adjust according to raw scan code/ASCII needs

    default: return c;
}
}

/**
 * Converts an entire string character-by-character using the Slovenian mapping logic.
 */
String convertSlovenianString(const String &input) {
    String output = "";
    output.reserve(input.length());
    for (size_t i = 0; i < input.length(); i++) {
        output += mapToSlovenianLayout(input.charAt(i));
    }
    return output;
}

/**
 * Reads from USBSerial, collects characters, and executes actions on CR (\r).
 */
void handleSerialInput() {
    while (USBSerial.available() > 0) {
        char incomingChar = (char)USBSerial.read();

        // 1. Check for Carriage Return (CR)
        if (incomingChar == '\r') {

            // Action 1: Check if received string is "pwd"

```

```

        if (inputBuffer == "pwd") {
            USBSerial.println("Waiting for data");
            dataConvertSaveFlag = true;
        }
        // Action 2: Process and convert string if the flag is enabled
        else if (dataConvertSaveFlag) {
            String convertedData = convertSlovenianString(inputBuffer);
            USBSerial.println(convertedData);
        }

        // Reset the buffer for the next incoming payload
        inputBuffer = "";
    }
    // Ignore Line Feed (\n) characters commonly paired with CR (\r\n)
    else if (incomingChar != '\n') {
        inputBuffer += incomingChar;
    }
}

void setup() {
    USBSerial.begin();
}

void loop() {
    // Call the function inside loop()
    handleSerialInput();
}

```

Key Logic Breakdown

- **Buffer Collection (inputBuffer):** Accumulates characters non-blockingly using `USBSerial.available()`. Line Feed (`\n`) characters are ignored so both raw CR and standard CR+LF terminal configurations work reliably.
- **pwd Detection:** Matches "pwd" exactly upon receiving `\r`. It outputs "Waiting for data" and sets `dataConvertSaveFlag = true`.

- **Slovenian Remapping (mapToSlovenianLayout):** Performs character translation (swapping y/z and Y/Z) before sending the processed response back through `USBSerial.println()`.
-

*User prompt: Show me how to expand the character mapping function for the ESP32-S3 to support Slovenian UTF-8 special characters like č, š, and ž over serial. I think also the symbols above numbers are different on US keyboards? on slovenian the sequence on top is [without shift key] 1234567890'+ [and with shift key] !"#\$%&/()=?**

Response: Because UTF-8 characters like č, š, and ž require **2 bytes** in UTF-8 encoding (0xC4/0xC5 byte prefixes), the mapping function must return a `String` or multi-byte string literal rather than a single `char`.

Below is the expanded mapping function translating US ASCII keypresses into the corresponding Slovenian QWERTZ layout symbols and special characters.

Updated Mapping Code

```
#include <Arduino.h>

/**
 * Maps incoming US layout ASCII characters to the corresponding
 * character/symbol on the Slovenian (SI) layout.
 */
String mapUsToSlovenianLayout(char c) {
    switch (c) {
        // --- 1. Letter Swaps ---
        case 'y': return "z";
        case 'z': return "y";
        case 'Y': return "Z";
        case 'Z': return "Y";

        // --- 2. Slovenian Special Characters (UTF-8) ---
        // Mapped from standard US key locations ([ ; \ ' ])
        case '[': return "š"; // UTF-8: 0xC5 0xA1
        case '{': return "Š"; // UTF-8: 0xC5 0xA0
        case ';': return "č"; // UTF-8: 0xC4 0x8D
```

```

case ':': return "Č"; // UTF-8: 0xC4 0x8C
case '\\': return "Ž"; // UTF-8: 0xC5 0xBE
case '|': return "ž"; // UTF-8: 0xC5 0xBD
case ']': return "đ"; // UTF-8: 0xC4 0x91
case '}': return "Đ"; // UTF-8: 0xC4 0x90
case '\\': return "ć"; // UTF-8: 0xC4 0x87
case '"': return "Ć"; // UTF-8: 0xC4 0x86

// --- 3. Unshifted Top Row (1234567890'+) ---
case '-': return "'"; // US '-' key position -> SI '\''
case '=': return "+"; // US '=' key position -> SI '+'

// --- 4. Shifted Top Row (!"#$%&/()=?*) ---
case '@': return "\""; // US Shift+2 -> SI Shift+2 (")
case '^': return "&"; // US Shift+6 -> SI Shift+6 (&)
case '&': return "/"; // US Shift+7 -> SI Shift+7 (/)
case '*': return "("; // US Shift+8 -> SI Shift+8 ((
case '(': return ")"; // US Shift+9 -> SI Shift+9 ())
case ')': return "="; // US Shift+0 -> SI Shift+0 (=)
case '_': return "?"; // US Shift+_ -> SI Shift+' (?)
case '+': return "*"; // US Shift+= -> SI Shift++ (*)

default:
    return String(c);
}
}

/**
 * Iterates through the input buffer and builds a UTF-8 compliant Slovenian string.
 */
String convertSlovenianString(const String &input) {
    String output = "";
    // Reserve extra space for multi-byte UTF-8 additions
    output.reserve(input.length() * 2);

```

```

for (size_t i = 0; i < input.length(); i++) {
    output += mapUsToSlovenianLayout(input.charAt(i));
}
return output;
}

```

Layout Mapping Reference

Input Key (US Layout)	Transformed Output (SI Layout)	UTF-8 Byte Sequence
[/ {	š / Š	0xC5 0xA1 / 0xC5 0xA0
; / :	č / Č	0xC4 0x8D / 0xC4 0x8C
\ /	ž / Ž	0xC5 0xBE / 0xC5 0xBD
Shift + 2 (@)	"	0x22
Shift + 6 (^)	&	0x26
Shift + 7 (&)	/	0x2F
Shift + 8 (*)	(	0x28
Shift + 9 ((	)	0x29
Shift + 0 (')	=	0x3D
Shift + - (_)	?	0x3F
Shift + = (+)	*	0x2A

*Note: Make sure your serial terminal emulator (e.g., PuTTY, Serial Monitor) is set to **UTF-8 encoding** to correctly render č, š, and ž.*

User prompt: this section has incorrect input and output, must be swapped. slovenian in, US output: // --- 3. Unshifted Top Row (1234567890'+) --- case '-': return '/'; // US '-' key position -> SI '/' case '=': return '+'; // US '=' key position -> SI '+' // --- 4. Shifted Top Row (!#\$%&/()=?*) --- case '@': return '\'; // US Shift+2 -> SI Shift+2 (') case '^': return '&'; // US Shift+6 -> SI Shift+6 (&) case '&': return '/'; // US Shift+7 -> SI Shift+7 (/) case '*': return '('; // US Shift+8 -> SI Shift+8 (() case '(': return ')'; // US Shift+9 -> SI Shift+9 ()) case ')': return '='; // US Shift+0 -> SI Shift+0 (=) case '_': return '?'; // US Shift+_ -> SI Shift+' (?) case '+': return '*'; // US Shift+= -> SI Shift++ (*)

Response: Here is the corrected code section with the inputs and outputs swapped so that Slovenian character inputs map to their corresponding US keyboard layout physical key positions:

```
// --- 3. Unshifted Top Row (Slovenian input -> US key position) ---
case '\': return "-"; // SI '\' (key 11) -> US '-'
case '+':  return "="; // SI '+' (key 12) -> US '='

// --- 4. Shifted Top Row (Slovenian input -> US key position) ---
case '@': return "\""; // SI '@' (Shift+2) -> US '"'
case '&': return "^"; // SI '&' (Shift+6) -> US '^'
case '/': return "&"; // SI '/' (Shift+7) -> US '&'
case '(': return "*"; // SI '(' (Shift+8) -> US '*'
case ')': return "("; // SI ')' (Shift+9) -> US '('
case '=': return ")"; // SI '=' (Shift+0) -> US ')'
case '?': return "_"; // SI '?' (Shift+') -> US '_'
case '*': return "+"; // SI '*' (Shift++) -> US '+'
```

Mapping Logic Summary

Received SI Character	Key Position	Converted US Output
'	Top row key 11	-
+	Top row key 12	=
"	Shift + 2	@
&	Shift + 6	^

Received SI Character	Key Position	Converted US Output
/	Shift + 7	&
(	Shift + 8	*
)	Shift + 9	(
=	Shift + 0	)
?	Shift + key 11	_
*	Shift + key 12	+

User prompt: add a timeout to the serial data reception, if there is no new character received for 10 seconds, clear the buffer.

Response: Here is the updated implementation using non-blocking timing with `millis()` to clear the buffer after 10 seconds of inactivity.

Updated Code Implementation

```
#include <Arduino.h>

// Global variables
bool dataConvertSaveFlag = false;
String inputBuffer = "";

// Timeout tracking variables
unsigned long lastReceiveTime = 0;
const unsigned long SERIAL_TIMEOUT_MS = 10000; // 10 seconds

/**
 * Reads from USBSerial, collects characters, handles CR execution,
```

```

* and clears the buffer if inactive for 10 seconds.
*/
void handleSerialInput() {
    // 1. Process incoming characters from USBSerial
    while (USBSerial.available() > 0) {
        char incomingChar = (char)USBSerial.read();

        // Update timestamp whenever a new character arrives
        lastReceiveTime = millis();

        // Check for Carriage Return (CR)
        if (incomingChar == '\r') {

            if (inputBuffer == "pwd") {
                USBSerial.println("Waiting for data");
                dataConvertSaveFlag = true;
            }
            else if (dataConvertSaveFlag) {
                String convertedData = convertSlovenianString(inputBuffer);
                USBSerial.println(convertedData);
            }

            // Reset buffer after execution
            inputBuffer = "";
        }
        // Ignore Line Feed (\n)
        else if (incomingChar != '\n') {
            inputBuffer += incomingChar;
        }
    }

    // 2. Inactivity Timeout Check
    // If the buffer contains partial data and 10 seconds have elapsed without new input, clear it
    if (inputBuffer.length() > 0 && (millis() - lastReceiveTime >= SERIAL_TIMEOUT_MS)) {
        inputBuffer = "";
    }
}

```

```
        // Optional debug prompt:  
        // USBSerial.println("[Timeout] Incomplete input cleared.");  
    }  
}
```

How It Works

1. **Timestamp Update (lastReceiveTime):** Every time `USBSerial.read()` processes a new character, `lastReceiveTime` updates to the current system runtime using `millis()`.
2. **Non-Blocking Timer Check:** On every execution of `handleSerialInput()`, the function checks if `inputBuffer` has partial data and if `millis() - lastReceiveTime >= 10000`. Unsigned integer subtraction safely handles `millis()` timer rollovers every ~49 days.
3. **Buffer Flush:** If 10 seconds pass without receiving a character, `inputBuffer` resets to "" without blocking main loop execution or losing the `dataConvertSaveFlag` state.

Verification

To verify this step, open your serial terminal, type a partial command (e.g., `pw` without pressing Enter), wait 10 seconds, and then type `d` followed by Enter. The command will fail or do nothing because `pw` was discarded after 10 seconds, leaving only `d` in the buffer.
